

第 1 章

开始

内容

1.1 编写一个简单的 C++ 程序	2
1.2 初识输入输出	5
1.3 注释简介	8
1.4 控制流	10
1.5 类简介	17
1.6 书店程序	21
小结	23
术语表	23

本章介绍 C++ 的大部分基础内容：类型、变量、表达式、语句及函数。在这个过程中，我们会简要介绍如何编译及运行程序。

在学习完本章并认真完成练习之后，你将具备编写、编译及运行简单程序的能力。后续章节将假定你已掌握本章中介绍的语言特性，并将更详细地解释这些特性。

2 学习一门新的程序设计语言的最好方法就是练习编写程序。在本章中，我们将编写一个程序来解决简单的书店问题。

我们的书店保存所有销售记录的档案，每条记录保存了某本书的一次销售的信息（一册或多册）。每条记录包含三个数据项：

0-201-70353-X 4 24.99

第一项是书的 ISBN 号（国际标准书号，一本书的唯一标识），第二项是售出的册数，最后一项是书的单价。有时，书店老板需要查询此档案，计算每本书的销售量、销售额及平均售价。

为了编写这个程序，我们需要使用若干 C++ 的基本特性。而且，我们需要了解如何编译及运行程序。

虽然我们还没有编写这个程序，但显然它必须

- 定义变量
- 进行输入和输出
- 使用数据结构保存数据
- 检测两条记录是否有相同的 ISBN
- 包含一个循环来处理销售档案中的每条记录

我们首先介绍如何用 C++ 来解决这些子问题，然后编写书店程序。

1.1 编写一个简单的 C++ 程序

每个 C++ 程序都包含一个或多个函数（function），其中一个必须命名为 **main**。操作系统通过调用 **main** 来运行 C++ 程序。下面是一个非常简单的 **main** 函数，它什么也不干，只是返回给操作系统一个值：

```
int main()
{
    return 0;
}
```

一个函数的定义包含四部分：返回类型（return type）、函数名（function name）、一个括号包围的形参列表（parameter list，允许为空）以及函数体（function body）。虽然 **main** 函数在某种程度上比较特殊，但其定义与其他函数是一样的。

在本例中，**main** 的形参列表是空的（**()** 中什么也没有）。6.2.5 节（第 196 页）将会讨论 **main** 的其他形参类型。

main 函数的返回类型必须为 **int**，即整数类型。**int** 类型是一种内置类型（built-in type），即语言自身定义的类型。

函数定义的最后部分是函数体，它是一个以左花括号（curly brace）开始，以右花括号结束的语句块（block of statements）：

3 {
 return 0;
}

这个语句块中唯一的一条语句是 **return**，它结束函数的执行。在本例中，**return**

还会向调用者返回一个值。当 `return` 语句包括一个值时，此返回值的类型必须与函数的返回类型相容。在本例中，`main` 的返回类型是 `int`，而返回值 `0` 的确是一个 `int` 类型的值。



请注意，`return` 语句末尾的分号。在 C++ 中，大多数 C++ 语句以分号表示结束。它们很容易被忽略，但如果忘记了写分号，就会导致莫名其妙的编译错误。

在大多数系统中，`main` 的返回值被用来指示状态。返回值 `0` 表明成功，非 `0` 的返回值的含义由系统定义，通常用来指出错误类型。

重要概念：类型

类型是程序设计最基本的概念之一，在本书中我们会反复遇到它。一种类型不仅定义了数据元素的内容，还定义了这类数据上可以进行的运算。

程序所处理的数据都保存在变量中，而每个变量都有自己的类型。如果一个名为 `v` 的变量的类型为 `T`，我们通常说“`v` 具有类型 `T`”，或等价的，“`v` 是一个 `T` 类型变量”。

1.1.1 编译、运行程序

编写好程序后，我们就需要编译它。如何编译程序依赖于你使用的操作系统和编译器。你所使用的特定编译器的相关使用细节，请查阅参考手册或询问经验丰富的同事。

很多 PC 机上的编译器都具备集成开发环境 (Integrated Developed Environment, IDE)，将编译器与其他程序创建和分析工具包装在一起。在开发大型程序时，这类集成环境可能是非常有用的工具，但需要一些时间来学习如何高效地使用它们。学习如何使用这类开发环境已经超出了本书的范围。

大部分编译器，包括集成 IDE 的编译器，都会提供一个命令行界面。除非你已经了解 IDE，否则你会觉得借助命令行界面开始学习 C++ 还是很容易的。这种学习方式的好处是，可以先将精力集中于 C++ 语言本身（而不是一些开发工具），而且，一旦你掌握了语言，IDE 通常是很容易学习的。

程序源文件命名约定

无论你使用命令行界面或者 IDE，大多数编译器都要求程序源码存储在一个或多个文件中。程序文件通常被称为源文件 (source file)。在大多数系统中，源文件的名字以一个后缀为结尾，后缀是由一个句点后接一个或多个字符组成的。后缀告诉系统这个文件是一个 C++ 程序。不同编译器使用不同的后缀命名约定，最常见的包括 `.cc`、`.cxx`、`.cpp`、`.cp` 及 `.C`。

4

从命令行运行编译器

如果我们正在使用命令行界面，那么通常是在一个控制台窗口内（例如 UNIX 系统中的外壳程序窗口或者 Windows 系统中的命令提示符窗口）编译程序。假定我们的 `main` 程序保存在文件 `prog1.cc` 中，可以用如下命令来编译它

```
$ CC prog1.cc
```

其中，`CC` 是编译器程序的名字，`$` 是系统提示符。编译器生成一个可执行文件。Windows 系统会将这个可执行文件命名为 `prog1.exe`。UNIX 系统中的编译器通常将可执行文件命名为 `a.out`。

为了在 Windows 系统中运行一个可执行文件，我们需要提供可执行文件的文件名，可

以忽略其扩展名 .exe:

```
$ prog1
```

在一些系统中,即使文件就在当前目录或文件夹中,你也必须显式指出文件的位置。在此情况下,我们可以键入

```
$ .\prog1
```

“.”后跟一个反斜线指出该文件在当前目录中。

为了在 UNIX 系统中运行一个可执行文件,我们需要使用全文件名,包括文件扩展名:

```
$ a.out
```

如果需要指定文件位置,需要用“.”后跟一个斜线来指出可执行文件位于当前目录中。

```
$ ./a.out
```

访问 main 的返回值的方法依赖于系统。在 UNIX 和 Windows 系统中,执行完一个程序后,都可以通过 echo 命令获得其返回值。

在 UNIX 系统中,通过如下命令获得状态:

```
$ echo $?
```

在 Windows 系统中查看状态可键入:

```
$ echo %ERRORLEVEL%
```

5

运行 GNU 或微软编译器

在不同操作和编译器系统中,运行 C++编译器的命令也各不相同。最常用的编译器是 GNU 编译器和微软 Visual Studio 编译器。默认情况下,运行 GNU 编译器的命令是 g++:

```
$ g++ -o prog1 prog1.cc
```

此处,\$是系统提示符。-o prog1 是编译器参数,指定了可执行文件的文件名。在不同的操作系统中,此命令生成一个名为 prog1 或 prog1.exe 的可执行文件。在 UNIX 系统中,可执行文件没有后缀;在 Windows 系统中,后缀为 .exe。如果省略了 -o prog1 参数,在 UNIX 系统中编译器会生成一个名为 a.out 的可执行文件,在 Windows 系统中则会生成一个名为 a.exe 的可执行文件(注意:根据使用的 GNU 编译器的版本,你可能需要指定 -std=c++0x 参数来打开对 C++11 的支持)。

运行微软 Visual Studio 2010 编译器的命令为 cl:

```
C:\Users\me\Programs> cl /EHsc prog1.cpp
```

此处,C:\Users\me\Programs>是系统提示符,\Users\me\Programs 是当前目录名(即当前文件夹)。命令 cl 调用编译器,/EHsc 是编译器选项,用来打开标准异常处理。微软编译器会自动生成一个可执行文件,其名字与第一个源文件名对应。可执行文件的文件名与源文件名相同,后缀为 .exe。在此例中,可执行文件的文件名为 prog1.exe。

编译器通常都包含一些选项,能对有问题的程序结构发出警告。打开这些选项通常是一个好习惯。我们习惯在 GNU 编译器中使用 -Wall 选项,在微软编译器中则使用 /W4。

更详细的信息请查阅你使用的编译器的参考手册。

1.1 节练习

练习 1.1: 查阅你使用的编译器的文档，确定它所使用的文件命名约定。编译并运行第 2 页的 main 程序。

练习 1.2: 改写程序，让它返回-1。返回值-1 通常被当作程序错误的标识。重新编译并运行你的程序，观察你的系统如何处理 main 返回的错误标识。

1.2 初识输入输出

C++语言并未定义任何输入输出（IO）语句，取而代之，包含了一个全面的**标准库**（standard library）来提供 IO 机制（以及很多其他设施）。对于很多用途，包括本书中的示例来说，我们只需了解 IO 库中一部分基本概念和操作。

本书中的很多示例都使用了 **iostream** 库。iostream 库包含两个基础类型 **istream** 和 **ostream**，分别表示输入流和输出流。一个流就是一个字符序列，是从 IO 设备读出或写入 IO 设备的。术语“流”（stream）想要表达的是，随着时间的推移，字符是顺序生成或消耗的。

标准输入输出对象

6

标准库定义了 4 个 IO 对象。为了处理输入，我们使用一个名为 **cin**（发音为 see-in）的 istream 类型的对象。这个对象也被称为**标准输入**（standard input）。对于输出，我们使用一个名为 **cout**（发音为 see-out）的 ostream 类型的对象。此对象也被称为**标准输出**（standard output）。标准库还定义了两个 ostream 对象，名为 **cerr** 和 **clog**（发音分别为 see-err 和 see-log）。我们通常用 cerr 来输出警告和错误消息，因此它也被称为**标准错误**（standard error）。而 clog 用来输出程序运行时的一般性信息。

系统通常将程序所运行的窗口与这些对象关联起来。因此，当我们读取 cin，数据将从程序正在运行的窗口读入，当我们向 cout、cerr 和 clog 写入数据时，将会写到同一个窗口。

一个使用 IO 库的程序

在书店程序中，我们需要将多条记录合并成单一的汇总记录。作为一个相关的，但更简单的问题，我们先来看一下如何将两个数相加。通过使用 IO 库，我们可以扩展 main 程序，使之能提示用户输入两个数，然后输出它们的和：

```
#include <iostream>
int main()
{
    std::cout << "Enter two numbers:" << std::endl;
    int v1 = 0, v2 = 0;
    std::cin >> v1 >> v2;
    std::cout << "The sum of " << v1 << " and " << v2
              << " is " << v1 + v2 << std::endl;
    return 0;
}
```

这个程序开始时在用户屏幕打印